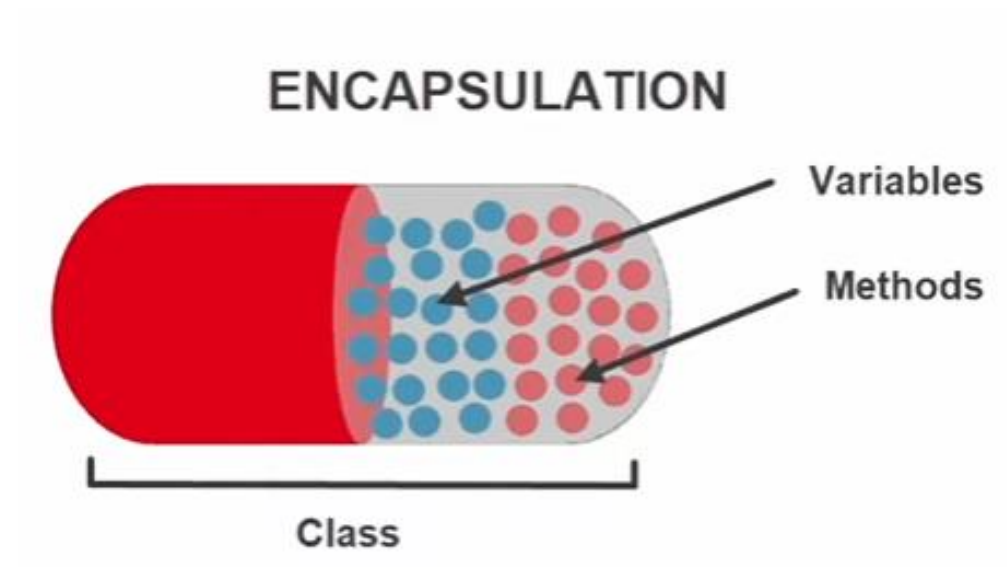


OOP Pillar : Encapsulation LECTURE

PREM KUMAR

Encapsulation

Wrapping up of data & information in a single unit, while controlling access to them.



Encapsulation

Wrapping up of data & information in a single unit, while controlling access to them.

```
class Car
{
    data members
    +
    methods (behavior)
}
```

ENCAPSULATION

Encapsulation

- Encapsulation is one of the fundamental principles of Object-Oriented Programming.
 - ❖ “Encapsulation hides the internal state and functionality of an object and only allows access through a public set of functions“
- Data hiding or Information Hiding is a Process in which we hide internal data from outside the world. The purpose of data hiding is to protect the data from misuse by the outside world. Data hiding is also known as data encapsulation. Without the Encapsulation Principle, we cannot achieve data hiding.

Implement Data Encapsulation

Data Encapsulation is implemented

- ❑ By declaring the **variables as private** (to restrict their direct access from outside the class)
- ❑ By defining one pair of public **setter and getter** methods or properties to access private variables from outside the class.

```
4 class YoutubeChannel{
5     public:
6     string Name;
7     string Owner;
8     int Subscriber;
9
10    YoutubeChannel(string name,string owner){
11        Name= name;
12        Owner= owner;
13        Subscriber=0;
14    }
15    void GetInfo(){
16        cout<<"name ="<<Name<<endl;
17        cout<<"owner="<<Owner<<endl;
18        cout<<"subscriber ="<<Subscriber<<endl;
19    }
20 }
21 };
22 int main() {
23     YoutubeChannel ytchannel("duet","seema");
24     ytchannel.GetInfo();
25 }
```

Output

```
name =duet
owner=seema
subscriber =0
```

It's not right!

```
22 ▾ int main() {  
23     YoutubeChannel ytchannel("duet","seema");  
24     ytchannel.Subscriber=1000;  
25     ytchannel.GetInfo();
```

Output

```
name =duet  
owner=seema  
subscriber =1000
```

```
private:
```

```
6 string Name;
```

```
7 string Owner;
```

```
8 int Subscriber;
```

```
public:
```

```
10 YoutubeChannel(string name,string owner){
```

```
11     Name= name;
```

```
12     Owner= owner;
```

```
13     Subscriber=0;
```

```
14 }
```

```
15 void GetInfo(){
```

```
16     cout<<"name ="<<Name<<endl;
```

```
17     cout<<"owner="<<Owner<<endl;
```

```
18     cout<<"subscriber ="<<Subscriber<<endl;
```

```
19 }
```

```
20 void subscriber()
```

```
{
```

```
    Subscribercount++;
```

```
}
```

```
void unsubscriber()
```

```
{
```

```
    Subscribercount--;
```

```
}
```

```
int main() {
```

```
    YoutubeChannel ytchannel("duet", "seema");
```

```
    ytchannel.subscriber();
```

```
    ytchannel.subscriber();
```

```
    ytchannel.subscriber();
```

```
    ytchannel.subscriber();
```

```
    ytchannel.GetInfo();
```

```
    ytchannel.unsubscriber();
```

```
    ytchannel.GetInfo();
```

```
    return 0;
```

```
}
```

Output

```
name = duet
```

```
owner = seema
```

```
subscriber = 4
```

```
*****
```

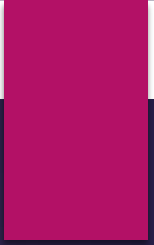
```
name = duet
```

```
owner = seema
```

```
subscriber = 3
```


Advantages of Encapsulation

- ❑ **Protection of data:** You can validate the data before storing it in the variable.
Achieving Data Hiding: The user will have no idea about the inner implementation of the class.
- ❑ **Security:** The encapsulation Principle helps to secure our code since it ensures that other units(classes, interfaces, etc) can not access the data directly.
- ❑ **Flexibility:** The encapsulation Principle in java makes our code more flexible, which in turn allows the programmer to change or update the code easily.
- ❑ **Control:** The encapsulation Principle gives more control over the data stored in the variables. For example, we can control the data by validating whether the data is good enough to store in the variable



Thank you